

LandScan AI — Audit & Completion Report

Intelligent Land Record Digitization & Validation System | Implementation Status

Executive Summary & Readiness Metrics

Metric	Value	Details / Formula
Total Modules Audited	321	Complete list across Categories A through S
■ Completed Modules	111	Fully implemented & functional without mock code
■ Partially Completed	18	UI or backend partially present, pending connection
■ Not Completed	192	Features currently missing from codebase
Overall Completion %	34.58%	$(111 / 321) \times 100$
Hackathon Readiness	Partially Ready	Core backend/GIS/audit working; OCR pipeline pending
Overall Audit Score	57 / 100	Weighted evaluation across 7 core categories

Module Breakdown by Category

Category	Total	Completed	Partial	Not Done
Category A — Project Foundation	10	10	0	0
Category B — Branding & Product Clarity	8	2	5	1
Category C — Authentication & Roles	10	6	3	1
Category D — Document Upload	20	9	2	9
Category E — Image Processing	10	0	0	10
Category F — OCR Integration	20	0	0	20
Category G — OCR Evaluation	10	0	0	10
Category H — Field Extraction	30	0	0	30
Category I — Confidence Scoring	15	0	0	15
Category J — Validation Engine	30	1	2	27
Category K — Human Review	24	0	0	24
Category L — Audit Trail	18	12	1	5
Category M — Dashboard	18	9	1	8
Category N — GIS	17	9	1	7
Category O — Analytics & Reports	12	4	0	8
Category P — API & Backend	19	12	0	7
Category Q — Data Integrity	16	10	1	5
Category R — UI/UX & Quality	20	18	2	0
Category S — Testing & Deployment	14	9	0	5

■ Completed Modules List (111 Modules)

These modules are fully implemented, functional, and integrated into the codebase with no manual database editing required.

ID	Category	Module / Requirement	Evidence Found
A01	A — Project Foundation	Project starts successfully	`app/main.py`, command `uvicorn app.main:app`
A02	A — Project Foundation	Frontend starts successfully	`app/main.py` line 51, `app/static/index.html`
A03	A — Project Foundation	Backend starts successfully	`app/main.py` line 37, 7 router modules included
A04	A — Project Foundation	Database or data layer starts successfully	`app/db.py`, `app/main.py` line 22
A05	A — Project Foundation	Environment variables are documented	`README.md` lines 13-14, `app/config.py`
A06	A — Project Foundation	README contains setup instructions	`README.md` lines 5-11
A07	A — Project Foundation	Required dependencies are listed	`requirements.txt` (fastapi, sqlalchemy, lightgbm)
A08	A — Project Foundation	Project has usable folder structure	Clean folder hierarchy
A09	A — Project Foundation	No blocking build errors	Server runs cleanly without errors
A10	A — Project Foundation	No major browser console errors	UI executes without console crashes
B07	B — Branding & Product Clarity	Government-tech visual identity is consistent	`app.css` color palette and card layouts
B08	B — Branding & Product Clarity	Navigation labels are clear	`TABS` array in `app.js` lines 132-136
C01	C — Authentication & Roles	Login screen exists	`index.html` `#login` section & `app.js` `FORMS.login`
C04	C — Authentication & Roles	Auditor / Viewer role exists	`security.py` supports `auditor` role
C05	C — Authentication & Roles	Role selection or authentication works	`auth.py` `/api/auth/login` and demo user chips
C06	C — Authentication & Roles	Role-specific dashboard works	`analytics.py` `scoped_projects` function
C07	C — Authentication & Roles	Unauthorized actions are blocked	`security.py` `require_roles` (10 tests in `test_auth_rbac.py`)
C09	C — Authentication & Roles	Viewer cannot edit records	`security.py` `require_roles` enforced on all write APIs
D01	D — Document Upload	PDF upload works	`documents.py` `write_version` endpoint
D02	D — Document Upload	PNG upload works	`documents.py` `write_version` endpoint
D03	D — Document Upload	JPG/JPEG upload works	`documents.py` `write_version` endpoint
D04	D — Document Upload	TIFF upload works	`documents.py` `write_version` endpoint
D07	D — Document Upload	File size validation works	`documents.py` line 86 checks max bytes (413 error)
D09	D — Document Upload	Document type can be selected	`documents.py` line 21 `CATEGORIES` list

D18	D — Document Upload	Uploaded file is actually stored	`documents.py` `write_version` writes to `STORAGE_DIR`
D19	D — Document Upload	Document ID is generated	`models.py` `Document.id` mapped column
D20	D — Document Upload	Document metadata is saved	`models.py` `DocumentVersion` table
J03	J — Validation Engine	Area must be positive rule works	`models.py` & `ParcelIn` `gt=0` constraint
L01	L — Audit Trail	Audit system exists	`app/audit.py` and `app/models.py` `AuditLog`
L02	L — Audit Trail	Document upload is logged	`documents.py` line 93 logs `document_uploaded`
L09	L — Audit Trail	User ID is logged	`AuditLog.actor` column in `models.py`
L10	L — Audit Trail	User role is logged	`AuditLog.role` column in `models.py`
L11	L — Audit Trail	Timestamp is logged	`AuditLog.ts` column in `models.py`
L12	L — Audit Trail	Previous value is logged	`AuditLog.detail` JSON dict stores `from` value
L13	L — Audit Trail	New value is logged	`AuditLog.detail` JSON dict stores `to` value
L14	L — Audit Trail	Comment is logged	`AuditLog.detail` JSON stores `note` field
L15	L — Audit Trail	Audit history is visible in UI	`app.js` `viewAudit` tab & `/api/audit` API
L16	L — Audit Trail	Audit records are linked to document	`AuditLog.entity_type` & `entity_id` columns
L17	L — Audit Trail	Audit records are not silently overwritten	`models.py` `AuditLog` append-only schema
L18	L — Audit Trail	Hash-chain or tamper-evident audit exists	`audit.py` `log` and `/api/audit/verify` endpoint
M01	M — Dashboard	Dashboard page exists	`app.js` `viewDashboard` function
M09	M — Dashboard	Processing-status chart works	`viewDashboard` in `app.js` & `charts.js`
M10	M — Dashboard	State-wise progress chart works	`analytics.py` `/api/dashboard/by-state` API
M11	M — Dashboard	District-wise progress works	`analytics.py` `/api/dashboard/trends` API
M14	M — Dashboard	Recent activity table works	`viewDashboard` early warnings table
M15	M — Dashboard	Dashboard filters work	`analytics.py` `summary` state & district params
M16	M — Dashboard	Dashboard data comes from actual data	`services/metrics.py` SQL aggregation
M17	M — Dashboard	Dashboard updates after actions	`app.js` re-invokes `viewDashboard` on hash change
M18	M — Dashboard	Dashboard is responsive	`app.css` grid breakpoints (`grid.g4`, `grid.g2`)
N01	N — GIS	GIS/map page exists	`map.js` `viewMap` function & `TABS` Map entry
N02	N — GIS	GeoJSON or GIS data is loaded	`analytics.py` `/api/gis/parcels` API endpoint
N04	N — GIS	Parcel polygons or markers are displayed	`map.js` `L.geoJSON` polygon layer rendering
N09	N — GIS	Parcel click interaction works	`map.js` `onEachFeature` click listener
N10	N — GIS	Parcel metadata popup works	`map.js` popup HTML template

N11	N — GIS	Parcel links to land record	`map.js` drawer trigger link
N13	N — GIS	State filter works	`analytics.py` `parcels_geojson` `state` param
N14	N — GIS	District filter works	`analytics.py` `parcels_geojson` `district` param
N16	N — GIS	Status filter works	`analytics.py` `parcels_geojson` `status` param
O01	O — Analytics & Reports	Analytics page exists	`app.js` `viewReports` tab
O10	O — Analytics & Reports	State/district report exists	`analytics.py` `/api/reports/mis` `group_by` param
O11	O — Analytics & Reports	Reports use actual project data	`analytics.py` `mis_report` SQL aggregation
O12	O — Analytics & Reports	Reports can be exported	`analytics.py` `format=csv` response writer
P01	P — API & Backend	Upload API exists	`documents.py` `upload_document` route
P02	P — API & Backend	Document-list API exists	`documents.py` `list_documents` route
P03	P — API & Backend	Document-detail API exists	`documents.py` `download_version` route
P11	P — API & Backend	Dashboard-summary API exists	`analytics.py` `summary` route
P12	P — API & Backend	GIS-parcel API exists	`analytics.py` `parcels_geojson` route
P13	P — API & Backend	Audit-history API exists	`system.py` `audit_list` route
P14	P — API & Backend	API responses have error handling	`HTTPException` handlers across all routers
P15	P — API & Backend	API inputs are validated	`ParcelIn`, `NotificationIn`, `AwardIn` models
P16	P — API & Backend	API outputs use consistent schemas	Standard dictionary serializers in `services/`
P17	P — API & Backend	Mock LRMS integration is clearly labelled	`integrations.py` docstring explicitly labels mock
P18	P — API & Backend	Mock DILRMP integration is clearly labelled	`integrations.py` docstring explicitly labels mock
P19	P — API & Backend	Mock GIS integration is clearly labelled	`README.md` & `models.py` docstring
Q01	Q — Data Integrity	Unique document IDs exist	`models.py` `Document.id` primary key
Q02	Q — Data Integrity	Unique record IDs exist	`models.py` `Parcel.id` primary key
Q09	Q — Data Integrity	Audit events link to documents	`AuditLog.entity_type="document"`, `entity_id`
Q10	Q — Data Integrity	GIS parcels link to records	`analytics.py` `parcels_geojson` includes `id`
Q11	Q — Data Integrity	Missing values are handled	`Optional` fields and `func.coalesce` in SQL
Q12	Q — Data Integrity	Duplicate data is handled	`UniqueConstraint("project_id", "stage")` etc.
Q13	Q — Data Integrity	Synthetic data is clearly labelled	`README.md` & login page demo text
Q14	Q — Data Integrity	Real sensitive personal data is not present	`app/seed.py` generates synthetic names
Q15	Q — Data Integrity	Data validation script exists	`tests/test_workflow.py` & `test_audit_documents.py`
Q16	Q — Data Integrity	Foreign-key checks exist	`ForeignKey(..., ondelete="CASCADE")` on tables

R01	R — UI/UX & Quality	Responsive desktop layout	`app.css` desktop layout rules
R02	R — UI/UX & Quality	Responsive tablet layout	`app.css` grid system
R04	R — UI/UX & Quality	Loading states exist	`app.js` line 157 skeleton card renderer
R05	R — UI/UX & Quality	Empty states exist	Empty banners and ` ` placeholder formatting
R06	R — UI/UX & Quality	Error states exist	`app.js` `toast(..., 'bad')` error handler
R07	R — UI/UX & Quality	Success notifications exist	`app.js` `toast(..., 'ok')` notification handler
R08	R — UI/UX & Quality	Forms show validation errors	`ApiError` message rendering on forms
R10	R — UI/UX & Quality	Tables are filterable	Filter parameters in `app.js`
R11	R — UI/UX & Quality	Buttons perform real actions	`ACT` event handler dictionary in `app.js`
R12	R — UI/UX & Quality	No broken links	Verified hash routing in `app.js`
R13	R — UI/UX & Quality	No broken images	Local Leaflet vendor assets in `static/vendor/`
R14	R — UI/UX & Quality	No major console errors	Clean console execution
R15	R — UI/UX & Quality	No major API errors	`pytest` test suite execution clean
R16	R — UI/UX & Quality	Colors are accessible	Checked color palette (`RISK_COLORS` & `STATUS_COLORS`)
R17	R — UI/UX & Quality	Status badges are consistent	`riskBadge` and `statusBadge` in `app.js`
R18	R — UI/UX & Quality	Typography is readable	`app.css` typography styling
R19	R — UI/UX & Quality	Navigation is consistent	`index.html` header and `startApp` tab builder
R20	R — UI/UX & Quality	No placeholder/lorem ipsum content	Realistic synthetic names generated in `seed.py`
S01	S — Testing & Deployment	Unit tests exist	36 test functions across 4 test files
S06	S — Testing & Deployment	Audit-log tests exist	`tests/test_audit_documents.py`
S07	S — Testing & Deployment	API tests exist	`tests/test_workflow.py` & `test_auth_rbac.py`
S09	S — Testing & Deployment	Test data is available	`app/seed.py` seeds demo database
S10	S — Testing & Deployment	Production/demo build works	`uvicorn app.main:app` works cleanly
S11	S — Testing & Deployment	Project can be run from README	Clear command instructions in `README.md`
S12	S — Testing & Deployment	Error logging exists	`logging.getLogger("uvicorn.error")` in `main.py`
S13	S — Testing & Deployment	Backup/export option exists	`/api/reports/mis` CSV exporter
S14	S — Testing & Deployment	Deployment instructions exist	`Dockerfile` & `docker-compose.yml`

■ Partially Completed Modules (18 Modules)

These modules are partially implemented (e.g. backend exists but UI is missing, or UI exists but uses placeholder values).

ID	Module / Requirement	Evidence Found	What Is Missing
B01	LandScan AI name is displayed	`index.html` & `main.py` display "LandScan"	Missing "AI" suffix
B02	Product subtitle is displayed	`index.html` displays "Land Acquisition & Management System"	Subtitle text does not match prompt requirement
B04	Problem statement is explained	Login card text in `index.html` line 14	Detailed problem statement panel missing
B05	Solution summary is explained	Login card subtitle	Solution architecture diagram/panel missing
B06	Synthetic-data disclaimer is visible	Generic demo note in `index.html` line 22	Exact required legal disclaimer text missing
C02	Admin role exists	`security.py` supports `central` role	Named `central` instead of `Admin`
C03	Revenue Officer / Verifier role exists	`security.py` supports `district` and `field` roles	Role not explicitly named `Revenue Officer / Verifier`
C10	Admin can manage configuration	`workflow.py` templates & system endpoints	Admin configuration panel UI missing
D10	State can be selected	Inherited from parent project state	Dedicated state dropdown on upload form missing
D11	District can be selected	Inherited from parent project district	Dedicated district dropdown on upload form missing
J01	Validation rules are loaded	Stage gate rules in `workflow.py` line 161	Land record validation rules missing
J02	Required fields are checked	Enforced in Pydantic models for parcel addition	Land record extraction checks missing
L06	Field correction is logged	`land.py` line 117 logs `parcel_updated`	Record review correction event missing
M02	Total documents metric works	Displays total projects & parcels	OCR document count KPI missing
N03	Synthetic GIS disclaimer is visible	Login demo note mentions synthetic coordinates	Map header explicit disclaimer badge missing
Q03	Documents link to records	Linked to `project_id`	Direct link to specific `parcel_id` missing
R03	Mobile-friendly layout	Functional, minor horizontal scroll on wide tables	Mobile drawer optimizations needed
R09	Tables are searchable	Filter dropdowns present	Full text search input missing

■ Not Completed Modules Overview (192 Modules)

The following major feature clusters are currently absent and require development for the full Land Record Digitization & Validation workflow:

- **1. Image Preprocessing (Category E — 10 Modules):** Grayscale, denoising, contrast boost, deskewing, binarization using OpenCV.

- **2. OCR Integration (Category F & G — 30 Modules):** PaddleOCR / Tesseract integration, OCR confidence capture, bounding box storage, raw text viewer UI.
- **3. Land Record Field Extraction (Category H — 30 Modules):** Regex / NLP extractors for Khasra, Khata, Patta, Owner Name, Survey No, Area (ha/sq.m), Village, District, Land Use, Mutation/Registration dates.
- **4. Confidence Scoring (Category I — 15 Modules):** Field-level confidence computation, color-coded badges (Green/Yellow/Red), and auto-routing trigger.
- **5. Land Record Validation Engine (Category J — 27 Modules):** Rule checks for Survey format, Khasra format, Duplicate parcel detection, 100% Share sum check, and GIS vs text area mismatch.
- **6. Human Review Queue (Category K — 24 Modules):** Review Queue page (`/review`), side-by-side document image preview vs extracted fields, Approve, Reject, and Field Correction actions.
- **7. Validation & Accuracy Analytics (Category O & M — 16 Modules):** OCR accuracy reports by language and validation error frequency charts.

Action Plan & Priority Fix Roadmap

- **P0-1: Branding & Synthetic Disclaimer:** Update header title to 'LandScan AI – Intelligent Land Record Digitization & Validation System' and display persistent legal disclaimer.
- **P0-2: OCR Integration Engine:** Build `app/services/ocr\_service.py` to process uploaded PDF/image files.
- **P0-3: Field Extractor:** Build `app/services/extractor.py` to extract Khasra, Khata, Patta, Owner, Area, Village, District, and Dates.
- **P0-4: Land Validation Engine:** Build `app/services/land\_validator.py` executing rules for Survey format, Khasra format, Duplicate check, 100% Share sum check, and GIS vs text area mismatch.
- **P0-5: Human Review Queue:** Build `/review` queue frontend view in `app.js` and side-by-side verification drawer in `drawer.js` with Approve, Reject, and Field Correction handlers.